

개계 [5a] 0] 매값등는] 레에그무용매 문용는 구드 [f.i 구만용원 a 구만그무매원 녀마
t 드용원매매도 값되구만원[드구만매매

Cgx갓mx검On값검In고DR고값InWbox검mx검In
3 Cx결값7: 4758A4: 0747:
C A , -
C,M- 3

K검DL KT Oec W
2
"ec W H" 0"ec W
" 2ec W 8 ,ad_X ag0ePX uX KgMS0ePX uRd_i a- 0WXX 0
0 2
C
52ad_X ag Cec W WXX
72ePX uX KgMS C
82ePX uRd_i a C
92 1 CWXX
CP관려고매결 " ec W " OK검DL "ec W KT
" 2K검DL 0ePX uX KgMS ePX uRd_i a
OP관려고매결 2

N7C
a] [f
,e게결x게결가 ,i 검게결x게결가 C
,B4 -C
C 0 0
C 0
C 0
ec W
, B4(-C
C 0 0
C 0
C 0
C 0
C

문제: 비구조화 데이터의 "의미" 분석
전통 SQL:
SELECT * FROM reviews WHERE review_text LIKE '%좋음%'
→ 정확한 텍스트 일치만 가능

→ "좋아", "훌륭함", "최고" 등 유사한 표현을 못 찾음

필요한 분석:

```
SELECT * FROM reviews
```

```
WHERE is_positive(review_text) = TRUE
```

→ 의미적으로 "긍정적"인 리뷰를 찾아야 함

→ 정확한 텍스트와 무관하게 "의미"를 이해해야 함

52단계가 필요! ec W C

사용자: "이 이미지와 분위기가 비슷한 제품을 찾아줘"

기존 접근 (Text-to-SQL):

1. 사용자의 자연어를 SQL로 변환

→ "분위기가 비슷하다"를 SQL WHERE 절로 표현하기 불가능

2. 부득이 원래 의도와 다른 SQL 생성

→ 사용자 불만족

이유: SQL 자체가 "정확한 일치"에만 적합

→ "의미적 유사성" 표현 불가

72 OL C

벡터 DB (예: Milvus):

```
SELECT * FROM products
```

```
WHERE embedding.similarity(query_embedding) > 0.8
```

```
LIMIT 10;
```

→ Top-k 유사도 검색은 가능

→ 하지만 "평균 가격"이나 "카테고리별 그룹핑" 같은
관계형 연산과의 결합이 자연스럽게 않음

필요한 쿼리:

```
SELECT category, AVG(price), COUNT(*) as count
```

```
FROM products
```

```
WHERE embedding.similarity(query_embedding) > 0.8
```

```
GROUP BY category
```

```
HAVING count > 5;
```

→ 벡터 DB에서는 이런 복합 분석이 어려움 (SQL 미지원)

82 C

현재 실무:

1. SQL DB에서 데이터 추출

2. Python에서 텍스트 전처리

3. Hugging Face에서 임베딩 생성

4. Milvus에 벡터 삽입

5. Milvus에서 검색 실행

6. 결과를 SQL로 다시 분석

7. 시각화

→ 6단계가 필요! 유지보수 난제

N O roi

KDL ec W 8 0" " ec W 2

COnpmk ns 4 i k 5

CmSPdP VWX 0 2

-- 기존 (불가능):

```
SELECT * FROM reviews
```

```
WHERE review_is_positive = TRUE;
```

-- AnDB (가능):

```
SELECT * FROM reviews
WHERE PROMPT('이 리뷰가 긍정적인가?', content) = 'positive';
```

C

데이터:

review_id	content	PROMPT 결과
1	"정말 좋음"	→ LLM 호출 → "이 리뷰 긍정적?" ← "positive"
2	"별로네"	→ LLM 호출 → "이 리뷰 긍정적?" ← "negative"
3	"괜찮음"	→ LLM 호출 → "이 리뷰 긍정적?" ← "neutral"

결과:
긍정적인 리뷰: 1개, 부정적: 1개, 중립: 1개

C

```
-- 질의응답 자동 처리
SELECT customer_id, COUNT(*) as question_count
FROM customer_messages
WHERE PROMPT('이것이 질문인가?', message) = 'yes'
GROUP BY customer_id;

-- 부정적 피드백 자동 탐지
SELECT customer_id, message
FROM reviews
WHERE PROMPT('이 리뷰에 불만이 있는가?', content) = 'yes'
AND PROMPT('긴급한 이슈인가?', content) = 'yes';

-- 지정된 언어의 메시지만 추출
SELECT * FROM messages
WHERE PROMPT('이 메시지가 한국어인가?', text) = 'yes';
```

1 C
WX 방 KaT
C544 0544 WX
Rag19 544
방 WX , -
DOr bk 과 [s_e 4 5
C ec W 2

```
-- 기존 (Vector DB):
-- Milvus에서 별도로 벡터 검색 수행

-- AnDB (SQL에서 직접):
SELECT * FROM products
WHERE SEM_MATCH(description, '가볍고 휴대하기 편한 노트북') > 0.8;
```

C

입력:

description	SEM_MATCH
"가벼운 울트라북"	embedding →
"15인치 화면"	유사도 계산 → 0.85 ✓
"무거운 데스크탑"	(threshold 0.8 초과)

결과: 첫 번째 상품만 반환

C

```
-- 중복 상품 찾기
SELECT a.product_id, b.product_id
FROM products a, products b
WHERE a.product_id < b.product_id
AND SEM_MATCH(a.description, b.description) > 0.85;

-- 유사한 고객 찾기
SELECT u1.user_id, u2.user_id, SEM_MATCH(u1.preference, u2.preference) as similarity
FROM users u1, users u2
WHERE SEM_MATCH(u1.preference, u2.preference) > 0.7
AND u1.user_id < u2.user_id;

-- 관련 뉴스 그룹화
SELECT news_id, SEM_MATCH(title, '인공지능 규제') as relevance
FROM news_articles
WHERE SEM_MATCH(title, '인공지능 규제') > 0.6;
```

EOr bk 과 pmt n 4 5

C 2

```
-- 기존 (불가능):
SELECT * FROM products
GROUP BY semantic_cluster; -- SQL에는 이런 함수 없음

-- AnDB (가능):
SELECT SEM_GROUP(description, 3) as cluster,
       AVG(price) as avg_price,
       COUNT(*) as count
FROM products
GROUP BY cluster;
```

C

입력 상품들:

- "가벼운 울트라북 - 1kg" (카테고리 내부적으로는 "경량 노트북")
- "얇은 모바일 워크스테이션 - 0.9kg" (역시 "경량 노트북")
- "게이밍 데스크탑 - 8kg" (다른 카테고리)
- "12인치 태블릿 - 500g" ("경량 기기")
- "휴대용 외장 HDD - 300g" ("경량 기기")

SEM_GROUP(..., 3): 3개 클러스터로 그룹화

클러스터 1 (경량 노트북): [1, 2]
클러스터 2 (경량 기기): [4, 5]
클러스터 3 (무거운 장비): [3]

결과:

cluster	avg_price	count	
1	1,500,000	2	(경량 노트북)
2	300,000	2	(경량 기기)
3	2,000,000	1	(무거운 장비)

C

```
-- 제품 카테고리를 설명의 의미에 따라 자동으로 재분류
SELECT
  SEM_GROUP(description, 10) as semantic_category,
  AVG(price) as avg_price,
  MIN(rating) as min_rating,
  COUNT(*) as product_count
FROM products
GROUP BY semantic_category
```

```
HAVING COUNT(*) > 5
ORDER BY avg_price DESC;

-- 고객의 선호도를 의미 기반으로 클러스터링
SELECT
    SEM_GROUP(preference, 5) as preference_cluster,
    COUNT(*) as customer_count,
    AVG(lifetime_value) as avg_ltv
FROM customers
GROUP BY preference_cluster;
```

NG

C

```
-- 긍정적인 리뷰를 가진 상품들을 의미 기반으로 분류
SELECT
    p.product_id,
    p.name,
    SEM_GROUP(p.description, 5) as product_cluster,
    COUNT(*) as positive_review_count,
    AVG(r.rating) as avg_rating
FROM products p
JOIN reviews r ON p.product_id = r.product_id
WHERE PROMPT('이 리뷰가 긍정적인가?', r.content) = 'yes'
AND SEM_MATCH(r.content, '품질이 좋다') > 0.7
GROUP BY p.product_id, p.name, product_cluster
HAVING COUNT(*) > 10
ORDER BY avg_rating DESC;
```

C

1. 모든 리뷰를 PROMPT로 감정 분석
↓ (필터링: 긍정적인 리뷰만)

2. 남은 리뷰를 SEM_MATCH로 품질 관련 여부 판정
↓ (필터링: '품질 좋음' 언급한 리뷰만)

3. 상품을 SEM_GROUP으로 의미 기반 클러스터화
↓

4. 클러스터별로 평균 평점 계산 및 정렬
↓

결과: 품질 좋은 제품들을 의미 기반 카테고리 분류

NG : 4. 리뷰별 군비매매법 클러스터링

O

검

C

기존 옵티마이저:
- I/O 비용만 고려
- 목표: 최소 비용 계획 선택

AnDB 옵티마이저:
- 1. LLM API 호출 비용 (토큰 수 × 가격)
- 2. 추론 정확도 (모델 크기, 프롬프트 품질)
- 3. 실행 시간 (대기 시간 포함)
- 목표: 예산 내에서 최대 정확도 계획 선택

O

```
SELECT * FROM reviews
WHERE PROMPT('이 리뷰가 긍정적인가?', content) = 'yes';
```

C

[illegible]

쿼리: 100만 리뷰 감정 분석, 예산 \$100,000	
옵티마이저의 고민:	
옵션 1: 모든 리뷰에 GPT-3.5 (비용: \$25,000, 정확도: 82%)	↳ 문제: "명백히 긍정" "명백히 부정"을 선별리 판단
옵션 2: 2단계 필터링	↳ 1단계: 로컬 모델로 "명백한" 사례들 필터링 (비용: \$0) ↳ 결과: 100만 중 50만 개로 축소 (명백한 긍정/부정) ↳ 2단계: GPT-3.5로 남은 50만 개 분석 (비용: \$12,500) ↳ 총 비용: \$12,500, 정확도: ~90% (1단계 로컬의 확신도 반영)
옵션 3: 비율 기반 샘플링	↳ 예산 배분: 저비용 모델 50%, 고비용 모델 50% ↳ 모든 리뷰를 저비용 모델로 빠르게 스캔 ↳ 그중 일부를 고비용 모델로 정밀 검증 ↳ 총 비용: \$100,000, 정확도: ~92%
옵티마이저 선택: 옵션 2	
→ 비용은 최소, 정확도는 충분하고, 자신감도 높음	

현재 (Exqtur의 세상):

- SQL이 기본 언어

- 벡터 검색은 SELECT 조건으로 표현
- LLM은 애플리케이션에서 호출

미래 (AnDB의 세상):

- SQL이 기본 언어이지만 시맨틱 토큰 포함
- 벡터 검색은 SEM_MATCH로 직접 표현
- LLM은 SQL에서 PROMPT로 호출
- 옵티마이저가 비용-정확도 균형

이 미래에서 Exqutor의 역할:

1. SEM_MATCH의 카디널리티 추정
 - "이 쿼리 결과가 몇 개일까?"를 정확히 예측
2. SEM_GROUP의 클러스터 크기 추정
 - "각 클러스터에 몇 개 행이 들어갈까?"
3. PROMPT의 선택도 추정
 - "이 프롬프트 필터가 몇 %를 제거할까?"

결론: AnDB 같은 시스템이 발전할수록,
Exqutor 스타일의 정확한 카디널리티 추정이
더욱 필수적이 된다!

N7 :

npmk ns

문제: LLM의 응답이 항상 일관적이지 않음

같은 프롬프트, 같은 텍스트도 LLM에 따라 다른 답:

텍스트: "가격이 좀 비싸네요"

GPT-4: "negative" (불평 감지)

Llama-2: "neutral" (서술적 관찰로 해석)

로컬 모델: "positive" (어떤 모델인지에 따라)

결과:

- 같은 데이터, 같은 SQL
- 다른 LLM 선택 → 다른 결과 반환
- 옵티마이저가 "얼마나 많은 행을 필터링할지" 예측 불가

H

```
SELECT * FROM reviews
WHERE PROMPT('이것이 불평인가?', content) = 'yes'
AND price > 100000;
```

C

P관련개념 C

PROMPT 결과: 전체의 30% = 300,000개

price > 100000: 전체의 20% = 200,000개

결합 (AND): 300,000 * 200,000 / 1,000,000 = 60,000개

WX

C

실제 실행 결과:

- GPT-4: 35만개 (엄격한 정의)
- Llama: 25만개 (너그러운 정의)
- 로컬: 15만개 (제한적 정의)

±50%의 오차 발생!

N7

" "

O

C

52 WX

C

544

OWX KaT

"

544갱경

544

때544갱경

F 544044

F 7A

72KaT C
KaT 결계기행간
KaT

82 C
ad_X ag
C5444 때Rag19 F %2 X , !-

92 ,O기결계간행- C
KMD
: 2 C
ad_X ag

52 ePX uX KgMS C ,5444/- 0 "42A
H"

72 ePX uRd_ i a C8 : H 0

82 CKDL ad_X ag 0 H
92 ,Qx결계간행 CWX 2 0 WXX OOL

: 2 ,K고기 C" H" ,WXX " " -